NEHA SHARMA U24AI050 LAB 11

10 April 2026 16:22

1. Implementation of resource allocation graph (RAG), where a number of processes, resources exit in the state of a system.

Assignment - Resource Allocation Graph (RAG) Implementation

Aim

To implement a **Resource Allocation Graph (RAG)** for a system consisting of multiple processes and resources, and to represent allocation and request relationships.

Theory

A **Resource Allocation Graph (RAG)** is a directed graph used in Operating Systems to represent the state of resource allocation in a system.

Components of RAG:

1. **Processes (P):**
   - Represented by circles
   - Example: P0, P1, P2
2. **Resources (R):**
   - Represented by rectangles
   - Example: R0, R1

Edges in RAG:

- **Request Edge (P → R):**
  Indicates that a process is requesting a resource.
- **Allocation Edge (R → P):**
  Indicates that a resource is allocated to a process.

Deadlock Condition:

- If the graph contains a **cycle**, then there is a **possibility of deadlock**.
- If there is **no cycle**, the system is in a safe state.

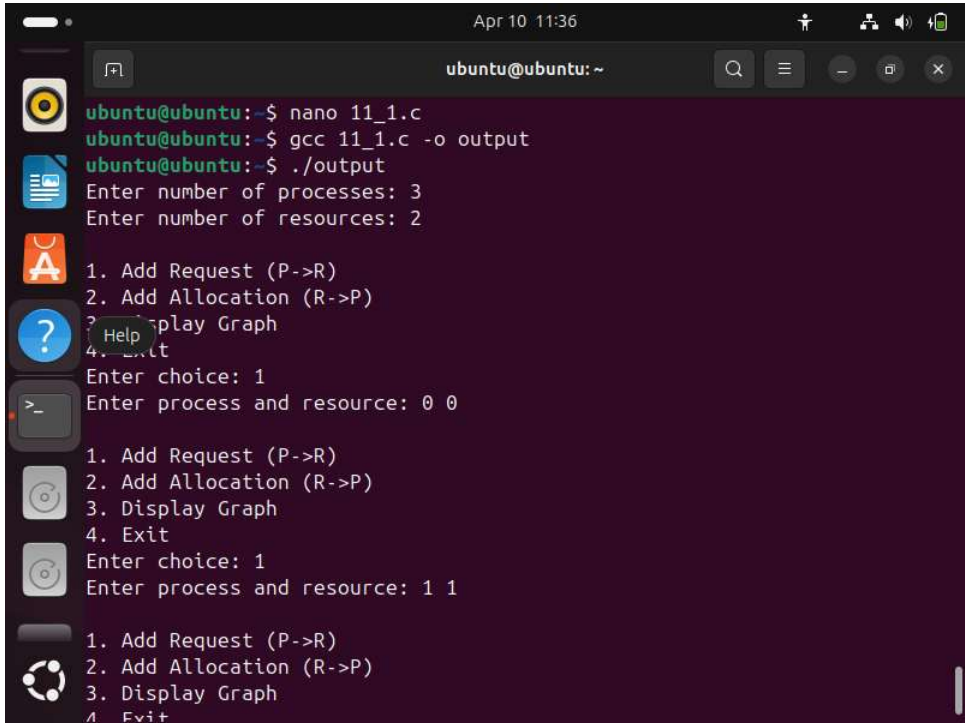
Algorithm

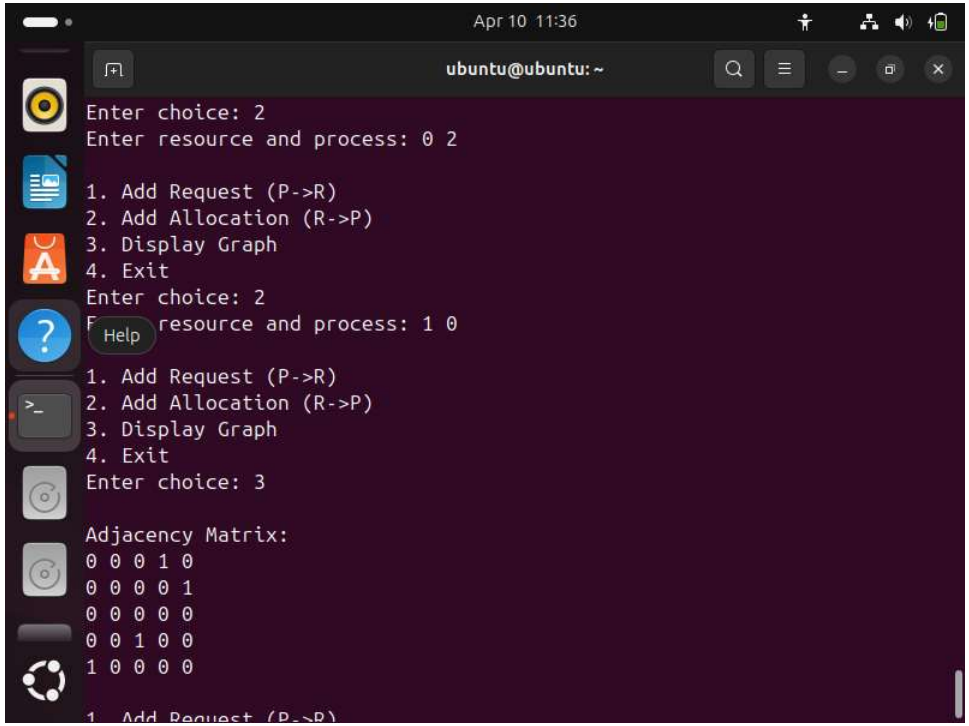
1. Input number of processes and resources
2. Create a graph using adjacency matrix

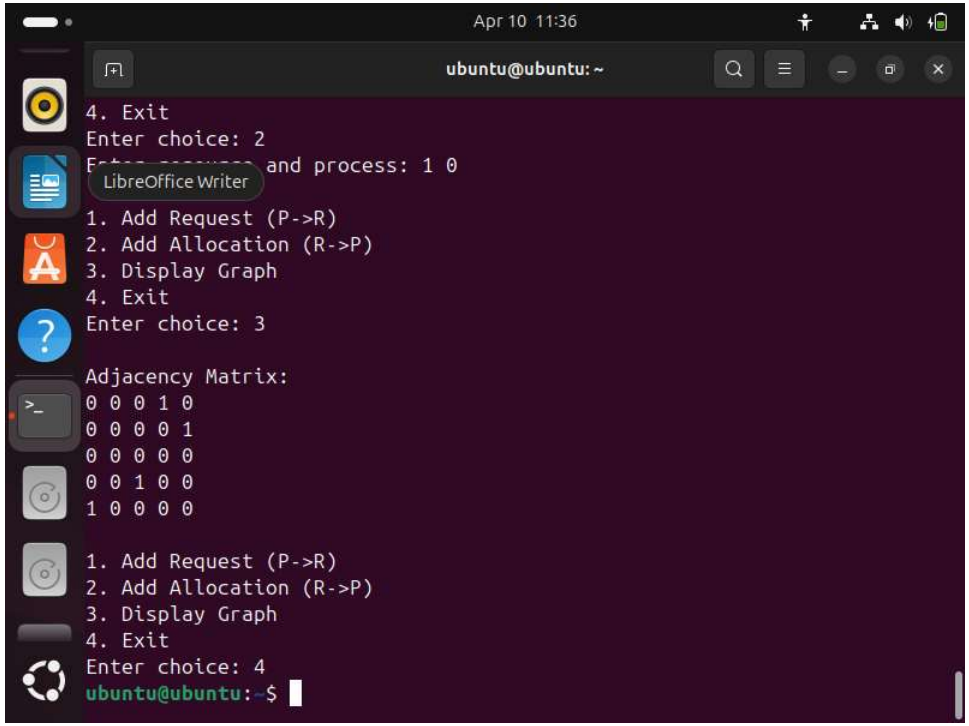
3. Add edges:
    - For request: Process => Resource
    - For allocation: Resource => Process
4. Display the graph

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int totalNodes;

// Function to add edge
void addEdge(int from, int to) {
   graph[from][to] = 1;
}

// Function to display graph
void display() {
   printf("\nAdjacency Matrix:\n");
   for (int i = 0; i < totalNodes; i++) {
      for (int j = 0; j < totalNodes; j++) {
         printf("%d ", graph[i][j]);
      }
      printf("\n");
   }
}

int main() {
   int p, r;

   printf("Enter number of processes: ");
   scanf("%d", &p);

   printf("Enter number of resources: ");
   scanf("%d", &r);

   totalNodes = p + r;

   // Initialize graph
```

```
    for (int i = 0; i < totalNodes; i++)
        for (int j = 0; j < totalNodes; j++)
            graph[i][j] = 0;

    int choice, from, to;

    while (1) {
        printf("\n1. Add Request (P->R)");
        printf("\n2. Add Allocation (R->P)");
        printf("\n3. Display Graph");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {

        case 1:
            printf("Enter process and resource: ");
            scanf("%d %d", &from, &to);
            addEdge(from, p + to); // P → R
            break;

        case 2:
            printf("Enter resource and process: ");
            scanf("%d %d", &from, &to);
            addEdge(p + from, to); // R → P
            break;

        case 3:
            display();
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
        }
    }
}
```


Apr 10 11:36
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ nano 11_1.c
ubuntu@ubuntu:~$ gcc 11_1.c -o output
ubuntu@ubuntu:~$ ./output
Enter number of processes: 3
Enter number of resources: 2
1. Add Request (P->R)
2. Add Allocation (R->P)
Help
4. Exit
Enter choice: 1
Enter process and resource: 0 0
1. Add Request (P->R)
2. Add Allocation (R->P)
3. Display Graph
4. Exit
Enter choice: 1
Enter process and resource: 1 1
1. Add Request (P->R)
2. Add Allocation (R->P)
3. Display Graph

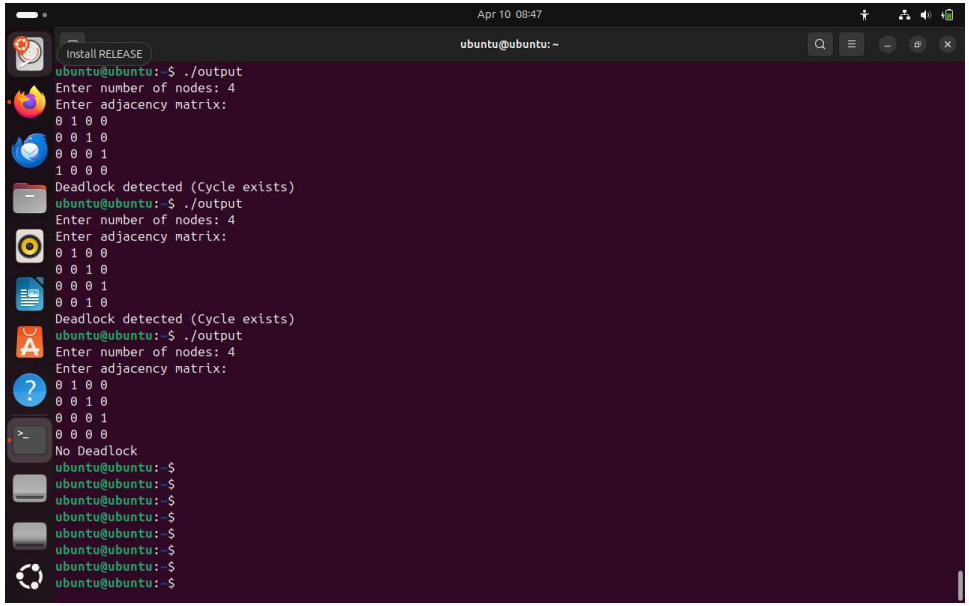

Apr 10 11:36
ubuntu@ubuntu: ~
Enter choice: 2
Enter resource and process: 0 2
1. Add Request (P->R)
2. Add Allocation (R->P)
3. Display Graph
4. Exit
Enter choice: 2
Help
resource and process: 1 0
1. Add Request (P->R)
2. Add Allocation (R->P)
3. Display Graph
4. Exit
Enter choice: 3
Adjacency Matrix:
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
0 0 1 0 0
1 0 0 0 0



2. Consider a state of system where there are number of process, number of resources. A process is requesting resource. Consider a scenario of a resource allocation graph or wait for graph with cycle and without cycle. Implement Cycle Detection Algorithm (DFS-based) in RAG to check for deadlock.

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 100

int graph[MAX][MAX];
bool visited[MAX], recStack[MAX];
int V; // total nodes

// DFS function
bool dfs(int node) {
   visited[node] = true;
   recStack[node] = true;

   for (int i = 0; i < V; i++) {
```

```
        if (graph[node][i]) {
            if (!visited[i] && dfs(i))
                return true;
            else if (recStack[i])
                return true;
        }
    }

    recStack[node] = false;
    return false;
}

// Check cycle
bool isDeadlock() {
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (dfs(i))
                return true;
        }
    }
    return false;
}

int main() {
    printf("Enter number of nodes: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    if (isDeadlock())
        printf("Deadlock detected (Cycle exists)\n");
    else
        printf("No Deadlock\n");

    return 0;
}
```



3. Write a program to implement a way to prevent runtime a possibility of a deadlock occurring in a system described as follows

Write a program with following specifications:

Command line input: name of a file

- The file contains the initial state of the system as given below (example included in right column)

#no of resources 4

#no of instances of each resource 2 4 5 3

#no of processes 3

#no of instances of each resource that 1 1 1 1, 2 3 1 2, 2 2 1 3

each processes needs in its lifetime.

The rule is that once a process uses a resource instance and returns it, the process no more needs it. For example, in the above set of 3 processes, If P2 finishes with 1 instance of resource r3, it will only need 3-1=2 instances of r3 in future. A process completes immediately when its need eventually reduces to 0.

Read this in

The programs then waits to accept a resource allocation or release requests-this is supplied through keyboard

For example:

0 a 1 0 1 1 indicates that p0 has requested allocation of 1 instance of R0, R2 and R3 each.

or

3 r 0 0 0 3 indicates that p3 is releasing 3 instances of resource r3
Your program should declare the result:
(1) should this request be granted? (i.e. guaranteeing that there will be no deadlock in future considering a worst remaining demand for resources)

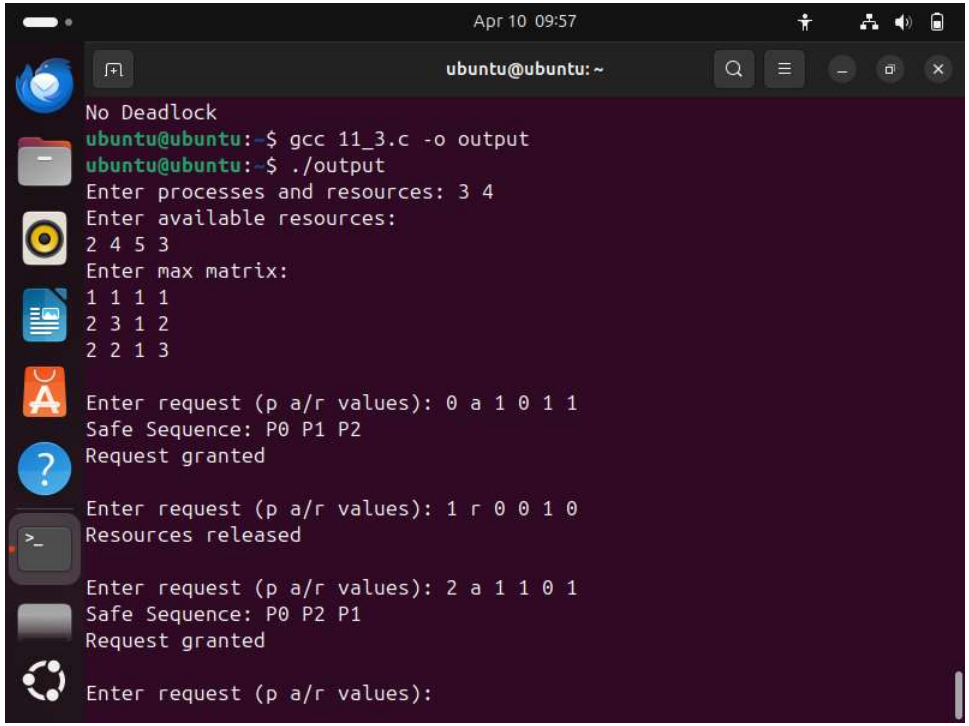
(2) if your answer is yes, print the safe sequence in which all remaining needs can be granted one by one and also grant the request, making necessary changes to system's state. If the requesting process's need is NIL, the program internally releases all its resources.
Go back to accept another request

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10

int available[MAX];
int max[MAX][MAX];
int allocation[MAX][MAX];
int need[MAX][MAX];

int n, m; // processes, resources

// Safety check
bool isSafe() {
   int work[MAX];
   bool finish[MAX] = {false};
   int safeSeq[MAX];

   for (int i = 0; i < m; i++)
      work[i] = available[i];

   int count = 0;

   while (count < n) {
      bool found = false;

      for (int i = 0; i < n; i++) {
         if (!finish[i]) {
            int j;
            for (j = 0; j < m; j++)
```

```
            if (need[i][j] > work[j])
                break;

            if (j == m) {
                for (int k = 0; k < m; k++)
                    work[k] += allocation[i][k];

                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
            }
        }
    }

    if (!found)
        return false;
}

printf("Safe Sequence: ");
for (int i = 0; i < n; i++)
    printf("P%d ", safeSeq[i]);
printf("\n");

return true;
}

// Request handling
void request(int p, int req[]) {

for (int i = 0; i < m; i++) {
    if (req[i] > need[p][i]) {
        printf("Error: Exceeds need\n");
        return;
    }

    if (req[i] > available[i]) {
        printf("Wait: Resources not available\n");
        return;
    }
}
```

```
    // pretend allocation
    for (int i = 0; i < m; i++) {
        available[i] -= req[i];
        allocation[p][i] += req[i];
        need[p][i] -= req[i];
    }

    if (isSafe()) {
        printf("Request granted\n");
    } else {
        printf("Request denied (unsafe)\n");

        // rollback
        for (int i = 0; i < m; i++) {
            available[i] += req[i];
            allocation[p][i] -= req[i];
            need[p][i] += req[i];
        }
    }
}

int main() {

    printf("Enter processes and resources: ");
    scanf("%d %d", &n, &m);

    printf("Enter available resources:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &available[i]);

    printf("Enter max matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j];
            allocation[i][j] = 0;
        }

    while (1) {
```

```
        int p, req[MAX];
        char type;

        printf("\nEnter request (p a/r values): ");
        scanf("%d %c", &p, &type);

        for (int i = 0; i < m; i++)
            scanf("%d", &req[i]);

        if (type == 'a') {
            request(p, req);
        } else {
            // release
            for (int i = 0; i < m; i++) {
                available[i] += req[i];
                allocation[p][i] -= req[i];
                need[p][i] += req[i];
            }
            printf("Resources released\n");
        }
    }
}
```


Apr 10 09:57
ubuntu@ubuntu: ~
No Deadlock
ubuntu@ubuntu:~$ gcc 11_3.c -o output
ubuntu@ubuntu:~$ ./output
Enter processes and resources: 3 4
Enter available resources:
2 4 5 3
Enter max matrix:
1 1 1 1
2 3 1 2
2 2 1 3
Enter request (p a/r values): 0 a 1 0 1 1
Safe Sequence: P0 P1 P2
Request granted
Enter request (p a/r values): 1 r 0 0 1 0
Resources released
Enter request (p a/r values): 2 a 1 1 0 1
Safe Sequence: P0 P2 P1
Request granted
Enter request (p a/r values):